

TianshangPulse 端侧房颤检测

模型训练与端到端部署技术报告

On-Device Atrial Fibrillation Detection: Training and Deployment Technical Report

TianshangPulse 团队

版本 v2.0 / 2026-09-06

摘要

本文档是 TianshangPulse —— 基于 ESP32-P4/RISC-V 的端侧 AI 智能手表固件——的房颤(Atrial Fibrillation, AF) 检测模型训练与端到端部署技术报告。系统以 MAX30102 PPG 为唯一生理信号源，在 400 MHz 双核 + 16 MB PSRAM 的约束下实现零云端依赖的健康监测（心率 / 血氧 / 异常事件，BLE GATT 与 TianshangHealth APP 双向同步）。

训练部分基于 MIMIC PERform AF 数据集（ODbL-1.0, 35 名受试者），经带通滤波、稀疏 NaN 插值、滑窗重采样与逐窗归一化后形成患者级隔离的 28 训练 / 7 验证受试者划分（10,500 个 4s 非重叠窗口 @100 Hz）。实测表明：端到端深度模型（CNN-LSTM, 226,145 参数）在小患者池上严重过拟合（验证 AUC 0.569），而搏动检测 + RR 间期统计特征 + 逻辑回归路线以仅 7 个参数在患者隔离验证集上达到 AUC 0.928、准确率 0.856、平衡准确率 0.868、F1 0.781。

部署部分给出完整的 PyTorch → ONNX → INT8 → TFLite Micro 流水线、固件内存布局、功耗预算、BLE 数据通路 & 离线事件缓存设计，并逐项标注现状与待补缺口。全流程固定随机种子，结果可在本仓库 scripts/ 下复现。

关键词：房颤检测；PPG 光电容积脉搏波；RR 间期；逻辑回归；端侧 AI；ESP32-P4；TFLite Micro；BLE GATT；MIMIC PERform AF

目录

1	项目概述	3
2	系统架构与资源约束	3
2.1	固件模块划分	3
2.2	内存布局	3
2.3	功耗预算与调度	4
3	数据准备	4
3.1	数据来源	4
3.2	预处理流水线	4
3.3	患者级数据划分	5

目录	2
4 特征工程：RR 间期统计	5
4.1 搏动检测	5
4.2 六维特征集	5
5 候选模型与训练设定	5
5.1 候选模型	5
5.2 CNN-LSTM 真实参数分账	6
5.3 训练配置与可复现性	6
6 实验结果与过拟合分析	6
6.1 最终模型：7 参数逻辑回归	6
6.2 三模型对比	7
6.3 过拟合机制分析	7
7 端到端部署流水线	7
7.1 现状盘点与缺口	7
8 固件集成与数据通路	7
8.1 推理引擎接口	7
8.2 BLE 数据通路 & 离线缓存	8
9 功耗与内存预算核对	8
10 结论与后续工作	8

1 项目概述

TianshangPulse是一套开源的端侧 AI 智能手表固件，运行于 **ESP32-P4**（400 MHz 双核 RISC-V，16 MB PSRAM / 16 MB Flash），与 Android 端 TianshangHealth APP 构成完全离线的健康监测生态。系统核心能力为：

- **端侧推理**：PPG波形经搏动检测与特征提取送入轻量分类器，推理延迟目标 < 10 ms，零云端依赖，全链路离线；
- **多生理指标**：实时心率、血氧，以及含房颤在内的异常事件检测；
- **传感器方案**：MAX30102（PPG，I2C）+ MPU6886（6轴 IMU，I2C）；
- **BLE 同步**：GATT Server（心率 / 血氧 / 异常事件 Notify + 模型 A/B 切换 + 批量同步），并发离线事件缓存。

固件支持 esp32p4 / esp32s3 双目标，本文以 ESP32-P4 为部署主目标。

2 系统架构与资源约束

2.1 固件模块划分

表 1: 固件模块（firmware/main/）

模块	文件	职责
入口	main.c	NVS 初始化、传感器/推理任务创建、模块装配
平台抽象	platform/	按 CONFIG_IDF_TARGET 分派的常量（P4/S3）
推理引擎	tflite/inference_engine.cc	TFLite Micro arena（PSRAM）管理、init/run/deinit
传感器	sensors/	统一接口 + MAX30102/MPU6886 驱动
BLE	ble/gatt_server.c	NimBLE GATT Server; offline_cache.c 离线事件环形缓冲
UI	ui/	LVGL v9 界面（骨架）
功耗	power/power_manager.c	esp_pm 调频、低功耗模式

2.2 内存布局

表 2: 关键资源分配（docs/MEMORY_LAYOUT.md）

数据	区域	大小	说明
TFLite arena	PSRAM	1 MB（KTensorArenaSize）	heap_caps_malloc(MALLOC_CAP_SPIRAM)
PPG 环形缓冲	SRAM	50 KB（KSensorBufferSize）	采样热路径
离线事件缓存	PSRAM	100 条（KMaxOfflineEvents）	循环缓冲区
任务栈	SRAM	sensor 4096 B / inference 8192 B	FreeRTOS

约束要点:ISR 内禁止堆分配;大块数据(模型/缓存)一律 PSRAM;sdkconfig.defaults.esp32p4 启用 120 MHz PSRAM 与 16 MB Flash。

2.3 功耗预算与调度

目标：主动监测 $\leq 30\text{ mA}$ ，待机 $\leq 500\text{ }\mu\text{A}$ （docs/POWER_BUDGET.md）。核心调度参数见 3。

表 3: 推理与采样调度参数（firmware/main/config.h）

常量	值	说明
KSensorSampleRateHz	100 Hz	PPG 采样率（与训练对齐）
KInferenceIntervalMs	40 ms	推理周期 25 Hz
KTensorArenaSize	1 MB	PSRAM arena
KBleMtuSize	512	BLE MTU
KMaxOfflineEvents	100	离线事件缓存容量

功耗激励：端侧 AF 检测若需连续采样与智能特征，必须将算力/内存开销压至极小——这是本报告选择特征工程路线的硬件前提。

3 数据准备

3.1 数据来源

训练数据集为 **MIMIC PERFORM AF Dataset¹**，含 35 名受试者（19 名 AF / 16 名非 AF），每人约 20 分钟胸骨 PPG，原始采样率 125 Hz。统计数据见表 4。

表 4: 数据集与预处理统计（data/processed/mimic_perform_af_meta.json）

项目	数值	备注
受试者总数	35	19 AF / 16 非 AF
原始采样率	125 Hz	目标采样率 100 Hz
窗口长度	4 s 非重叠	$100\text{ Hz} \times 4\text{ s} = 400\text{ 点}$
总窗口数	10,500	AF 5,700 / 非 AF 4,800
训练 / 验证窗口	8,400 / 2,100	28 训 / 7 验，患者零重叠
验证集类别占比	AF 28.6%	按患者划分所致

3.2 预处理流水线

scripts/prepare_af_dataset.py 按序执行：

- 1. 稀疏 NaN 插值：线性插值 + 首尾填充；缺失 $> 25\%$ 的受试者整体剔除；
- 2. 带通滤波：四阶 Butterworth 带通（0.5–8 Hz），去除基线漂移与高频噪声；
- 3. 滑窗与重采样：4 s / 步长 4 s 非重叠滑窗，线性插值由 125 Hz 重采样至 100 Hz（400 点），与固件 100 Hz 采样率对齐；
- 4. 逐窗归一化：每窗独立 Z-score（标准差下限 10^{-8} ），截断至 $[-3, 3]$ 。

¹记录号 10.5281/zenodo.15906524，ODbL-1.0。

3.3 患者级数据划分

GroupShuffleSplit (random_state=42, 验证 20%) 按受试者分组划分, 并断言训练/验证无患者重叠, 避免同一受试者连续波形跨集合泄漏。由于按患者划分, 训练集 (AF 60.7%) 与验证集 (AF 28.6%) 类别比例存在差异, 故报告额外给出平衡准确率与 F1。

4 特征工程：RR 间期统计

4.1 搏动检测

对每个归一化窗口 $s \in \mathbb{R}^{400}$ 做峰值检测 (scipy.signal.find_peaks, 最小峰距 $\lfloor 100 \times 0.4 \rfloor = 40$ 点即 400 ms, 对应上限约 150 bpm), 差分得到 RR 间期 (毫秒):

$$\text{RRI}_i = \frac{p_{i+1} - p_i}{f_s} \times 1000 \quad [\text{ms}], \quad f_s = 100 \text{ Hz}. \quad (1)$$

窗口内搏动数 < 3 时 RRI 特征不成立 (对应列置 0)。

4.2 六维特征集

设 RR 间期序列 $R = \{R_1, \dots, R_n\}$, 均值 μ 、标准差 σ ; 信号 s ; 窗长 (秒) T_w , 六维特征为:

$$\text{rri_std} = \sigma(R), \quad (2)$$

$$\text{rmssd} = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n-1} (R_{i+1} - R_i)^2}, \quad (3)$$

$$\text{pnn50} = \frac{1}{n-1} \sum_{i=1}^{n-1} \mathbb{I}(|R_{i+1} - R_i| > 50), \quad (4)$$

$$\text{cv} = \frac{\sigma(R)}{\mu(R)} \quad (\mu > 0), \quad (5)$$

$$\text{hr_proxy} = \frac{N_{\text{peaks}}}{T_w} \quad [\text{beats/s}], \quad (6)$$

$$\text{diff_std} = \sigma(\text{diff}(s)). \quad (7)$$

其中 rri_std/cv/rmssd/pnn50 刻画心律不齐与间期不稳定性 (房颤的典型生理标志), hr_proxy 提供速率上下文, diff_std 保留波形幅度信息。

5 候选模型与训练设定

5.1 候选模型

1. 逻辑回归 (LR, 最终采用): 6 维特征线性分类, 参数 $= 6 \times 1 + 1 = 7$;
2. MLP-32 (备选): $6 \rightarrow 32 \xrightarrow{\text{ReLU+Dropout}(0.2)} \rightarrow 1$, 参数 $= (6 \times 32 + 32) + (32 \times 1 + 1) = 224 + 33 = 257$;
3. CNN-LSTM (弃用): 卷积前端 + 2 层 LSTM + MLP 头, 共 226,145 参数, 架构细目见表 5。

5.2 CNN-LSTM 真实参数分账

此前报告版本误称该模型为「2 层 LSTM」。对存档 model/af/af_lstm.pth（状态字典，912,584 B）逐张量核账，其真实架构为 **Conv1d 前端 + BatchNorm × 2 + 2 层 LSTM + MLP 头**，见表 5。

表 5: CNN-LSTM 可训练参数分账（源自 af_lstm.pth 状态字典）

模块	层（张量）	形状	参数数
卷积前端	conv.0 Conv1d(1→16, k5)	(16,1,5)+bias	96
	conv.3 Conv1d(16→32, k5)	(32,16,5)+bias	2592
BatchNorm	conv.1 (BN, ch=16)	weight+bias	32
	conv.4 (BN, ch=32)	weight+bias	64
LSTM 第 1 层	lstm.weight_ih_l0	(4·128, 32)	16384
	lstm.weight_hh_l0	(4·128, 128)	65536
	lstm.bias_ih/hh_l0		1024
LSTM 第 2 层	lstm.weight_ih_l1	(4·128, 128)	65536
	lstm.weight_hh_l1	(4·128, 128)	65536
	lstm.bias_ih/hh_l1		1024
MLP 头	head.0 Linear(128→64)	(64,128)+bias	8256
	head.3 Linear(64→1)	(1,64)+bias	65
合计（含 BN 可训练参数，不含 BN 统计缓存）			226,145

对照验证：卷积 2688 + BN 96 + LSTM 三层门控 215,040 + MLP 头 8321 = 226,145。该数量与 af_lstm_report.json 记录的 226,145 一致，证明其统计正确；af_lstm.onnx 输入为 ppg[1,400,1]，说明模型直接消费原始波形。

5.3 训练配置与可复现性

所有实验固定 seed=42。LR 用 sklearn LogisticRegression (C=1.0,max_iter=5000)。MLP 用 Adam (lr=1e-3, weight_decay=1e-3)，批大小 64，最大 300 轮，按验证 AUC 早停 (patience 30) 并回滚最优权重，损失为 BCEWithLogits。CNN-LSTM 配置：hidden 128、2 层、dropout 0.1、批大小 128、最多 50 轮、patience 10。环境：Python 3.12.10 / PyTorch 2.5.1+cu124 / scikit-learn 1.9.0 / SciPy 1.13.1 / NumPy 1.26.4。

6 实验结果与过拟合分析

6.1 最终模型：7 参数逻辑回归

固定 seed 复现（LR 全流程约 0.1 s），结果见表 6。验证 AUC **0.928**，高于训练集 AUC 0.837——患者隔离验证比同源窗口更保守，而该差距较小证明特征稳定性良好。

表 6: 逻辑回归（7 参数）训练与患者隔离验证指标

数据集	AUC	准确率	平衡准确率	F1	灵敏度 / 特异度
训练	0.837	0.823	0.798	0.862	—
验证	0.928	0.856	0.868	0.781	0.895 / 0.841

验证混淆矩阵 $\mathbf{C}^{\text{LR}} = \begin{bmatrix} 1261 & 239 \\ 63 & 537 \end{bmatrix}$ （行 = 真实：非 AF / AF；列 = 预测：非 AF / AF， $n = 2100$ ）。导出灵敏度 = $537/600 = 0.895$ ，特异度 = $1261/1500 = 0.841$ ，精确率 = $537/776 = 0.692$ 。

6.2 三模型对比

表 7: 患者隔离验证集模型对比 (seed=42)

模型	参数数	AUC	准确率	平衡准确率	F1	状态
逻辑回归	7	0.928	0.856	0.868	0.781	最终采用
MLP-32	257	0.916	0.791	0.836	0.720	备选
CNN-LSTM	226,145	0.569	0.291	0.503	0.446	弃用

三个混淆矩阵对比：

$$\mathbf{C}^{\text{MLP}} = \begin{bmatrix} 1094 & 406 \\ 34 & 566 \end{bmatrix}, \quad \mathbf{C}^{\text{CNN-LSTM}} = \begin{bmatrix} 11 & 1489 \\ 1 & 599 \end{bmatrix}. \quad (8)$$

6.3 过拟合机制分析

CNN-LSTM (226K参数) 训练损失迅速压向 0，但验证特异度仅 0.007（几乎把所有样本判为 AF），AUC 0.569。其根因是：原始 PPG 波形蕴含巨大的个体间差异（肤色、血管状态、运动伪影、佩戴松紧），高容量模型沿训练患者即可近乎完美拟合而无真正学到房颤的泛化共性；35 名受试者的训练池不足以支撑端到端学习。

RR 间期统计天然去除个体波形指纹而保留房颤共性（绝对节律不规则），因而 7 参数 LR 在患者隔离验证下反超 226K 深度模型。这一结果与临床共识——房颤筛查依赖 RR 间期分析——一致。

7 端到端部署流水线

模型从训练到固件需要经过完整的转换与部署链路，见图 1。

PyTorch (FP32) → ONNX → INT8 (per-channel) → TFLite Micro → model_data.cc

图 1: 模型转换流水线 (AGENTS.md ML-Agent: FP32 权重 → 计算图 → 量化 → 固件加载)

7.1 现状盘点与缺口

说明：当前 `convert_model.py` 仅将 ONNX 字节数组转成 `model_data.cc`，真实量化 (INT8 per-channel + 校准) 与算子集校验 (LSTM/GRU resolver) 尚未实现；`inference_engine.run` 尚未搭 TFLite 解释器。这是部署的首要待办。

8 固件集成与数据通路

8.1 推理引擎接口

```
esp_err_t inference_engine_init(void);    // PSRAM 分配 1MB arena
```

表 8: 部署流水线现状 (v2.0)

环节	脚本 / 文件	状态
ONNX 导出 (LR)	model/af/af_model.onnx	✓LR 已导出 (features[1,6]→logit[1,1])
ONNX 导出 (CNN-LSTM)	model/af/af_lstm.onnx	✓已导出 (弃用存档)
PyTorch → ONNX → INT8	scripts/convert_model.py	✗ 占位实现 (仅生成字节数组)
INT8 校准	—	✗ 未执行 (需 ≥1000 样本)
余弦相似度校验	scripts/verify_model.py	✗ 占位 (TODO)
TFLite Micro 加载	tflite/inference_engine.cc	✗ arena 就绪, 推理未接入

```
esp_err_t inference_engine_run(void);    // 目前骨架: memset 结果
esp_err_t inference_engine_get_result(inference_result_t *out);
inference_result_t { heart_rate_bpm; blood_oxygen_pct;
                    anomaly_flag;    confidence; }
```

main.c 创建 sensor (优先级 6, 100 Hz) 与 inference (优先级 5, 25 Hz) 两个任务; 目前传感器驱动读数皆为零值占位, 尚未与推理数据通路打通。

8.2 BLE 数据通路与离线缓存

依据 docs/PROTOCOL.md, GATT 服务包含: 心率 0x2A37、血氧 0x2A5F (Notify), 用户配置 0xFFFF1、模型 A/B 切换 0xFFFF3 (Write), 异常事件 0xFFFF2 (Notify)、批量同步 0xFFFF4 (Read/Write)。字节序小端、CRC-8 校验。

AF 检测结果经 0xFFFF2 上报为异常事件 (类型 + 8 B 时间戳 + 置信度 0–100)。无网/未连接时写入 offline_cache.c 的 PSRAM 环形缓冲 (100 条, 满则覆盖最旧), 恢复连接后经 0xFFFF4 批量上报, APP 确认后清除。

9 功耗与内存预算核对

目标 (docs/POWER_BUDGET.md): 主动监测 ≤ 30 mA、待机 ≤ 500 μA。其可行性依赖: LP 核看门狗/唤醒 (P4 的 40 MHz 核), HP 核仅在推理时全速; PSRAM 空闲半休眠; 传感器单次采样; 推理 25 Hz × < 10 ms 占空比 ≈ 25%。

内存方面, 特征工程路线比 **226K 深度模型** 更契合约束: 分类器仅存 7 个系数, 而 CNN-LSTM 至少需要一个 226K × (INT8) ≈ 226 KB 的权重区 (占 arena 1 MB 的 22%) + 一层时序激活。LR 则几乎零权重占用。

10 结论与后续工作

结论

1. 数据: MIMIC PERform AF 10,500 窗, 28/7 患者级隔离, 杜绝受试者泄漏;
2. 方法: 端到端 CNN-LSTM (226,145 参数) 在小患者池上过拟合 (验证 AUC 0.569), 改采搏动检测 + 6 维 RRI 统计 + 逻辑回归;

3. 性能: 7参数实现验证 AUC **0.928**、平衡准确率 0.868、F1 0.781, 优于 257 参数 MLP; 固定 seed 严格复现;
4. 可部署: 全部运算是峰值检测与标量统计, 零模型权重, 契合 ESP32-P4 INT8/TFLite 约束。

后续工作 (按优先级)

1. `convert_model.py` 接入真实 INT8 per-channel 量化 (校准集 ≥ 1000 样本) 与 TFLite Micro 导出;
2. `verify_model.py` 完成余弦相似度 (> 0.95) 与字节数一致性校验, 生成 `benchmark.md`;
3. `inference_engine.cc` 接入 TFLite 解释器, `inference_result_t` 增加 AF 输出字段;
4. 固件实现峰值检测与 RRI 特征提取 (复用训练端同参数), 打通 `sensor`→`engine` 数据通路;
5. 数据增强与外部数据集 (含运动伪影) 鲁棒性验证。

参考文献

- [1] MIMIC PERFORM AF Dataset. Zenodo, record 10.5281/zenodo.15906524, ODbL-1.0. <https://zenodo.org/records/15906524>
- [2] Tianshang301. TianshangPulse: ESP32-P4 端侧 AI 智能手表固件. <https://github.com/Tianshang301/TianshangPulse>
- [3] Tianshang301. TianshangHealth: 配套 Android 健康监测 App. <https://github.com/Tianshang301/TianshangHealth>
- [4] Espressif. ESP-IDF Programming Guide (ESP32-P4). <https://docs.espressif.com/projects/esp-idf/en/latest/esp32p4/>
- [5] TensorFlow. TFLite Micro. <https://github.com/tensorflow/tflite-micro>